SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

Somaiya TRUST



# TRAEFIK

Zero-Config Service Discovery and Automatic HTTPS Routing for Containerized Deployments

*DevOps Theory IA: Case Study*

Submitted by

**Pradyum Mistry - 16010123236**

B.Tech Computer Engineering, Last Year

K. J. Somaiya School of Engineering, Mumbai

DevOps (216U01E744), Semester VII, 2026-2027

October 2026

Demo files: github.com/altf4-games/tugboat, folder traefik-demo

# Abstract

Traefik is an open-source reverse proxy and edge router built around containers. Instead of reading a list of backends from a file, it asks the platform what is running and builds its routes from that. For HTTPS it requests a certificate from a certificate authority the first time a hostname is used, and renews it before it expires. This case study looks at those two features: service discovery without configuration, and automatic HTTPS.

I explain how Traefik is structured, then show both features working. The first demonstration runs Traefik 3.7.14 with the Docker provider and adds, scales and removes apps while Traefik keeps running. The second runs a complete ACME exchange on my laptop against Pebble, Let's Encrypt's test server, and checks the certificate Traefik receives. I also describe how Traefik is used in my mini project, Tugboat, and compare it with nginx, Caddy, HAProxy and Envoy.

The demonstrations are Docker Compose files in the Tugboat repository, so they can be repeated. The main limits are that the HTTPS demonstration used a test certificate authority and not Let's Encrypt, and that I tested only Traefik myself.

**Keywords:** *Traefik, reverse proxy, service discovery, Docker provider, ACME, automatic HTTPS, Let's Encrypt, containers.*

# Table of contents

**1. Introduction....6**
1.1 Background....6
1.2 Problem statement....6
1.3 Objectives....6
1.4 Method....6
**2. Tool selection....7**
2.1 Where Traefik fits in the DevOps cycle....7
2.2 Why I chose Traefik....7
2.3 About the tool....7
**3. How Traefik works....8**
3.1 Core concepts....8
3.2 The path of a request....8
3.3 Static and dynamic configuration....9
3.4 Dashboard and API....9
**4. Zero-config service discovery....10**
4.1 The problem with static configuration....10
4.2 How the Docker provider works....10
4.3 The labels I used....11
4.4 Load balancing across replicas....11
4.5 Other providers....11
**5. Automatic HTTPS....12**
5.1 Why HTTPS needs a certificate authority....12
5.2 The ACME protocol....12
5.3 Challenge types....12
5.4 How Traefik runs the exchange....12
5.5 Limits of the approach....13
**6. Practical demonstration....14**
6.1 Environment....14
6.2 Demonstration 1: service discovery....14
6.3 Demonstration 2: automatic HTTPS....16
6.4 What the demonstration shows, and what it does not....17
6.5 Problems I met....18
**7. Traefik inside Tugboat....19**
7.1 Branch previews through the Docker provider....19
7.2 Production through the file provider....19
7.3 What Tugboat does not use Traefik for....19
**8. Comparison and evaluation....20**
8.1 The alternatives....20
8.2 Comparison....20
8.3 Evaluation....20
8.4 When I would choose which....21
**9. Conclusion....22**
**References....23**

# List of figures

Figure 1: Traefik in the DevOps cycle.....7
Figure 2: The path of a request through Traefik.....9
Figure 3: What happens when a container with Traefik labels starts.....10
Figure 4: The ACME exchange with the HTTP-01 challenge.....13
Figure 5: Dashboard: HTTP routers after two apps were started.....15
Figure 6: Dashboard: HTTP services, with three servers behind whoami.....15
Figure 7: Dashboard: the secure router with its TLS certificate resolver.....17
Figure 8: Dashboard: the certificate Traefik obtained.....17
Figure 9: How Tugboat uses Traefik.....19

# List of tables

Table 1: Traefik concepts and where they appear in my demo....8
Table 2: What each label does....11
Table 3: ACME challenge types in Traefik....12
Table 4: Demonstration environment....14
Table 5: Scope of the demonstration....17
Table 6: Comparison of Traefik with other proxies....20
Table 7: Choosing a proxy....21

# 1. Introduction

## 1.1 Background

A container gets a new internal IP address every time it starts, and containers start and stop all the time: a new version is deployed, a replica is added, a crashed process is restarted. Something has to sit in front of them, accept requests on a public port, and pass each request to the right container. That job belongs to a reverse proxy.

Traditional proxies such as nginx read their backends from a configuration file. When a container appears, someone edits the file and reloads the proxy, and when an address changes the file is wrong. HTTPS adds a second chore: every hostname needs a certificate that is requested, installed and renewed before it expires.

## 1.2 Problem statement

This case study looks at two problems that every container deployment runs into, and at how one tool, Traefik, solves both:

1. How does a proxy keep its routing table in step with the containers that are running, without anyone editing a file?
2. How does a proxy obtain, install and renew a TLS certificate for each hostname without anyone doing it by hand?

## 1.3 Objectives

1. Explain where Traefik sits in the DevOps cycle and why I chose it.
2. Describe how Traefik is built, how it discovers services, and how it obtains certificates through ACME.
3. Show both features working on real software, and record what I observed.
4. Describe how Traefik is used in my mini project, Tugboat.
5. Compare Traefik with nginx, Caddy, HAProxy and Envoy, and give an evaluation.

## 1.4 Method

I read the official Traefik documentation [1][2][3], then ran Traefik 3.7.14 on Docker 29.4.0 (OrbStack on macOS, Apple silicon) and recorded what happened. Both demonstrations are Docker Compose files in the `traefik-demo/` folder of the Tugboat repository [13]. The command output in this report is copied from a real run on 15 October 2026, with some lines left out for length. I ran only Traefik. The other tools in the comparison are described from their own documentation and announcements.

# 2. Tool selection

## 2.1 Where Traefik fits in the DevOps cycle

Figure 1 shows the usual stages of the DevOps cycle. Traefik works at the end of the loop. After a new version is deployed, Traefik is the part that sends live traffic to it, and while the application runs it keeps routing correctly as containers come and go.

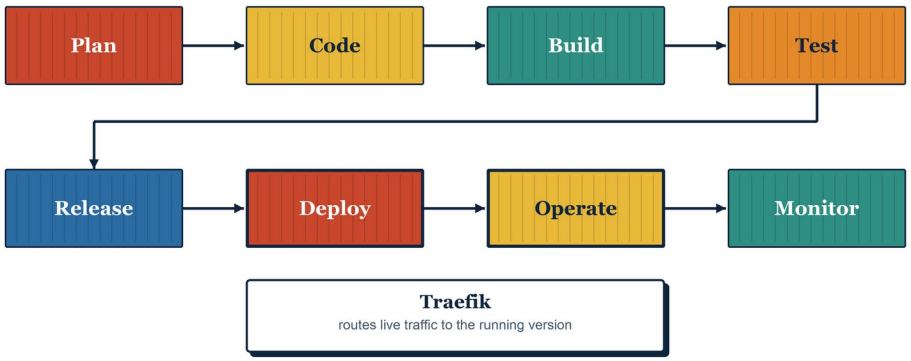



*Figure 1: Traefik in the DevOps cycle*

## 2.2 Why I chose Traefik

- It was designed for containers, so discovery from Docker is a core feature.
- It handles certificates itself, so both parts of my case study title live in one tool.
- I already use it in Tugboat for branch previews and for the production route, so I have real experience of where it works well and where it does not.
- It is free and open source, and both features can be tested on a laptop.

## 2.3 About the tool

Traefik began as a side project of Emile Vauge in 2015, and version 1.0 followed in 2016 [4]. It is written in Go and released under the MIT license, and it is developed by Traefik Labs, which was earlier called Containous [4]. It ships as a single binary and as a Docker image, and it includes a dashboard and a JSON API. This report uses version 3.7.14, the image Docker pulled on 15 October 2026.

# 3. How Traefik works

## 3.1 Core concepts

Traefik has six main concepts. The table gives each one with an example from my first demonstration.

*Table 1: Traefik concepts and where they appear in my demo*

| Concept | What it is | Example from the demo |
|---|---|---|
| **Entrypoint** | A network port that Traefik listens on | `web`, on port 80 inside the container |
| **Router** | Matches requests by rule and sends them to a service | `` `Host(whoami.127.0.0.1.nip.io)` `` |
| **Middleware** | Changes a request or response on the way through | A headers middleware that adds `X-Served-By` |
| **Service** | Where the request finally goes, usually a load balancer over several servers | The three `whoami` containers |
| **Provider** | A source of routing configuration | The Docker provider |
| **Certificate resolver** | How Traefik obtains certificates | `pebble`, an ACME server |

## 3.2 The path of a request

Figure 2 follows one request. It arrives on an entrypoint. Traefik tests it against the rules of its routers and picks the one that matches. The router's middlewares run in order, and then the router hands the request to its service, which chooses one of the containers behind it. The providers feed this whole structure from the side and can change it while requests are flowing.

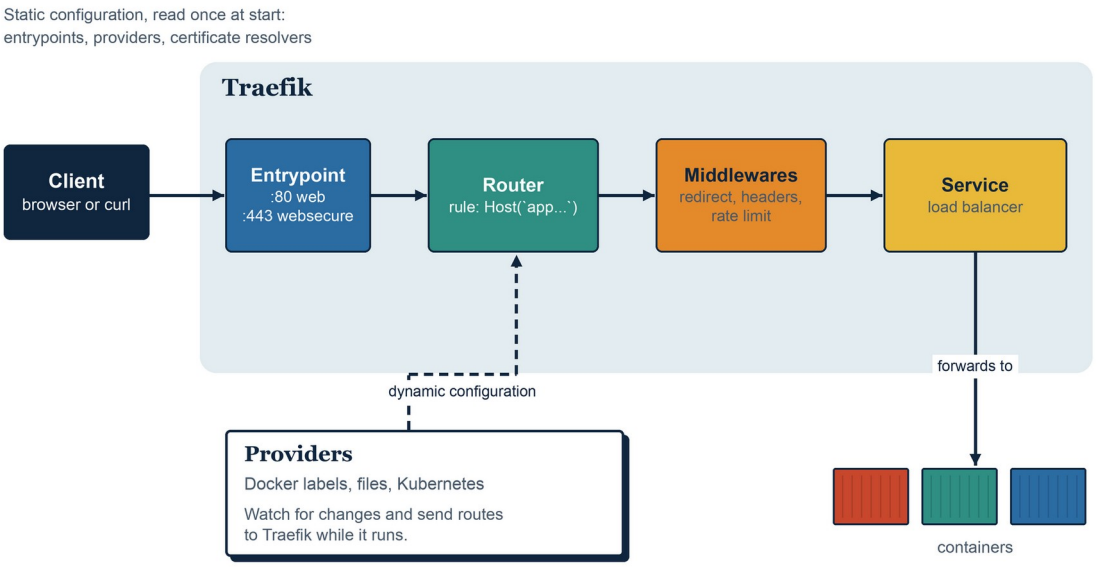



*Figure 2: The path of a request through Traefik*

## 3.3 Static and dynamic configuration

Traefik splits its configuration in two. The static configuration is read once when Traefik starts. It defines the entrypoints, which providers to use, and the certificate resolvers. Changing it needs a restart. The dynamic configuration describes the routers, services, middlewares and TLS options. Providers supply it, and Traefik applies every change without a restart.

In my demonstrations the static configuration is the `command:` list of the Traefik service in the Compose file. The dynamic configuration is entirely the labels on the other containers.

## 3.4 Dashboard and API

Traefik includes a dashboard and a JSON API on the same port. The dashboard lists every router, service and middleware that Traefik currently knows about, with a status for each. The API gives the same data as JSON, for example `GET /api/http/routers`, and Tugboat uses it to confirm that a route has loaded. Both demonstrations start Traefik with `--api.insecure=true`, which exposes them without authentication. That setting is only suitable for a local test.

# 4. Zero-config service discovery

## 4.1 The problem with static configuration

With a file-based proxy, each backend is written into the configuration. A minimal nginx example looks like this:

```
upstream whoami { server 192.168.117.3:80; }
server {
  listen 80;
  server_name whoami.example.test;
  location / { proxy_pass http://whoami; }
}
```

Every new container means another block, a syntax check and a reload, and the address `192.168.117.3` is only valid until the container is recreated. I did not run this example. It shows why the file has to change as often as the containers do.

## 4.2 How the Docker provider works

The Docker provider connects to the Docker daemon through its socket. It lists the running containers and then watches Docker's event stream, which it does by default [2]. Whenever a container starts or stops, the provider reads that container's labels and the details Docker reports about it, such as its IP address and exposed ports. From the labels that start with `traefik.` it builds routers, services and middlewares and hands them to Traefik. Figure 3 shows the sequence.

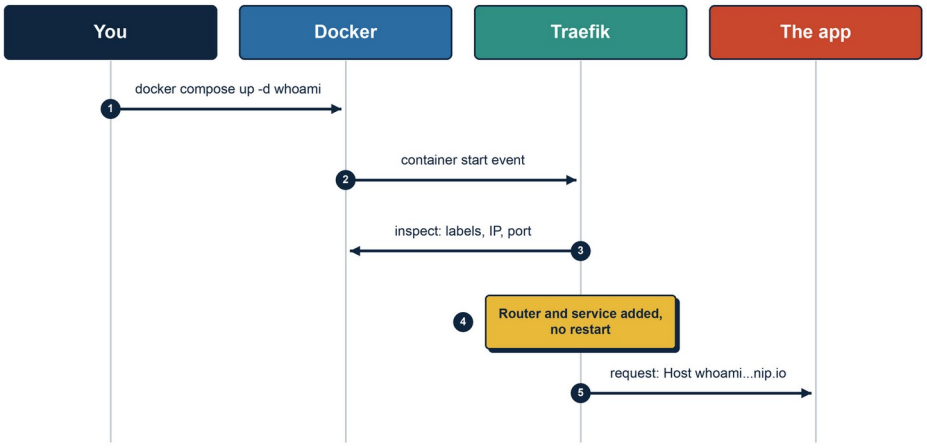



*Figure 3: What happens when a container with Traefik labels starts*

Two details from the documentation [2] matter in practice:

- By default Traefik exposes every container it can see. Setting `exposedByDefault` to false makes it ignore any container that lacks `traefik.enable=true`. My demonstrations use that setting.

- If a container exposes one port, Traefik uses it. If it exposes several, Traefik uses the lowest. When neither is right, the `loadbalancer.server.port` label names the port.

## 4.3 The labels I used

The first demonstration labels one container with five lines. They are the entire routing configuration for that app:

```
labels:
  - traefik.enable=true
  - traefik.http.routers.whoami.rule=Host(`whoami.127.0.0.1.nip.io`)
  - traefik.http.routers.whoami.entrypoints=web
  - traefik.http.routers.whoami.middlewares=served-by
  - traefik.http.middlewares.served-by.headers.customresponseheaders.X-Served-By=traefik
```

*Table 2: What each label does*

| Label | Effect |
|---|---|
| `traefik.enable=true` | Tells Traefik to manage this container, because `exposedByDefault` is off |
| `...routers.whoami.rule` | Creates a router named `whoami` that matches one hostname |
| `...routers.whoami.entrypoints` | Allows the router on the `web` entrypoint only |
| `...routers.whoami.middlewares` | Attaches the middleware named `served-by` |
| `...middlewares.served-by.headers...` | Defines that middleware, which adds a response header |

The hostname needs no DNS setup, because nip.io resolves any name of that form to the IP address written inside it.

## 4.4 Load balancing across replicas

When several containers carry the same router and service names, Traefik puts them behind one service and shares requests between them. I scaled the app to three replicas without touching Traefik, and six requests went to all three containers in turn (Section 6.2). Scaling down again removed the extra servers from the service.

## 4.5 Other providers

Docker is one provider among many. The file provider reads routes from YAML or TOML files and reloads them when they change, and Tugboat uses it for the production route. Other providers read Kubernetes resources, Docker Swarm services, and catalogs such as Consul. The router, service and middleware model is the same for all of them.

### A note on security

Reading Docker's events needs access to the Docker socket, and unrestricted access to that socket is access to the whole host. The Traefik documentation warns about this and suggests a socket proxy in front of Docker [2]. I mount the socket read-only in both demonstrations, which is acceptable on my laptop but not something I would do unchanged on a shared server.

# 5. Automatic HTTPS

## 5.1 Why HTTPS needs a certificate authority

A browser trusts an HTTPS site because the site presents a certificate signed by a certificate authority (CA) that the browser already trusts. Getting such a certificate by hand means generating a key, creating a signing request, proving to the CA that you control the domain, installing the result, and repeating the whole process before the certificate expires. It is easy to forget a renewal.

## 5.2 The ACME protocol

ACME, the Automatic Certificate Management Environment, turns that manual process into an API [5]. Let's Encrypt is the best-known CA that offers it. A client registers an account, places an order for a domain name, and completes a challenge that proves it controls the name. It then submits a certificate signing request and receives the signed certificate. Traefik contains an ACME client, so it performs these steps itself.

## 5.3 Challenge types

The challenge is the proof of control. Traefik supports three kinds [3].

*Table 3: ACME challenge types in Traefik*

| Challenge | What the CA checks | Requirement | Wildcards |
|---|---|---|---|
| **HTTP-01** | Fetches a token over plain HTTP from `/.well-known/acme-challenge/` | Traefik reachable from the CA on port 80 | No |
| **TLS-ALPN-01** | Makes a TLS connection and expects a special certificate | Traefik reachable from the CA on port 443 | No |
| **DNS-01** | Looks for a token in a DNS record | Traefik can create DNS records at a supported provider | Yes |

## 5.4 How Traefik runs the exchange

A certificate resolver is declared in the static configuration. It names the ACME server, an email address, the challenge to use, and a file that stores the results. A router then opts in with one label, `tls.certresolver=<name>`. When such a router appears for a hostname that has no certificate, Traefik runs the exchange in Figure 4 and starts serving HTTPS for that name as soon as the certificate arrives.

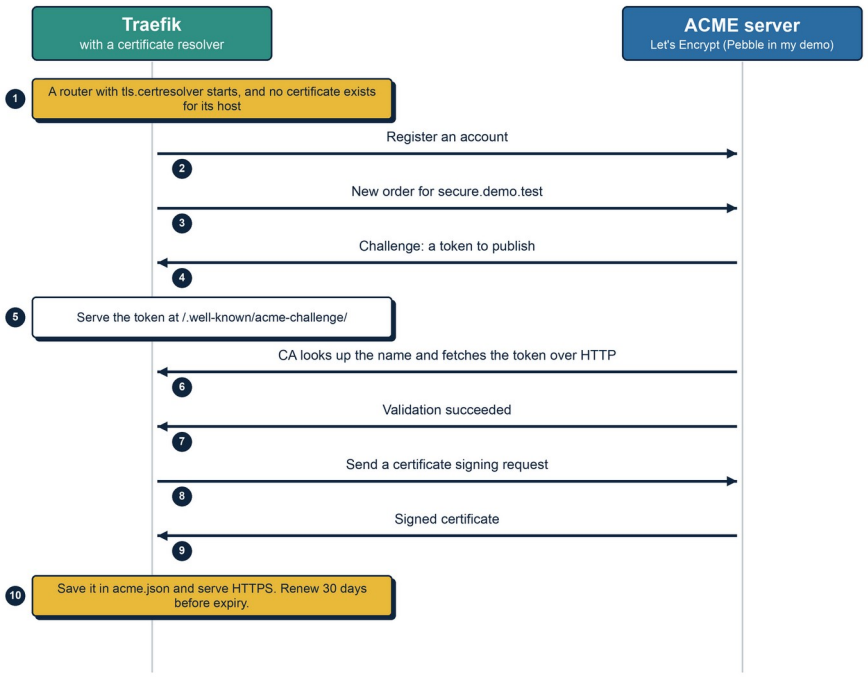



*Figure 4: The ACME exchange with the HTTP-01 challenge*

The results are kept in the storage file, which is `acme.json` by default. Traefik starts renewing a certificate 30 days before it expires, and the default certificate duration it assumes is 2160 hours, which is 90 days [3]. The default CA is the production Let's Encrypt server. The `caServer` option points Traefik at any other ACME server, which is how I used a local test server in my demonstration.

## 5.5 Limits of the approach

- A public CA must be able to reach Traefik on port 80 or 443, or be able to see a DNS record, so the name has to be a real public domain. A name that only exists on a private network needs the DNS challenge or a private CA.
- Wildcard certificates need the DNS challenge, and the DNS provider has to be one that Traefik supports [3].
- Public CAs enforce rate limits, so experiments should use a staging or test server first.
- The state lives in one file, so a single Traefik instance is the simple case.

# 6. Practical demonstration

## 6.1 Environment

*Table 4: Demonstration environment*

| Item | Value |
|---|---|
| **Machine** | MacBook, Apple silicon, macOS |
| **Docker** | 29.4.0, running on OrbStack |
| **Docker Compose** | 5.1.2 |
| **Traefik** | `traefik:v3.7.14` |
| **Demo apps** | `traefik/whoami`, a small server that prints details about each request |
| **ACME server** | Pebble and its challenge test server, from Let's Encrypt [6] |
| **Files** | `traefik-demo/discovery/` and `traefik-demo/https/` in the Tugboat repository |

## 6.2 Demonstration 1: service discovery

The first Compose file starts Traefik and nothing else. The apps are in a separate profile, so I can start them later while Traefik runs. After `docker compose up -d`, Traefik's API lists only its own two routers, and a request for an app hostname gets a 404:

```
$ curl -s localhost:8081/api/http/routers
['api@internal', 'dashboard@internal']
$ curl -s -o /dev/null -w "%{http_code}\n" -H "Host: whoami.127.0.0.1.nip.io"
localhost:8000/
404
```

Then I started the first app. Traefik was not touched:

```
$ docker compose --profile apps up -d whoami
$ curl -i -H "Host: whoami.127.0.0.1.nip.io" localhost:8000/
HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8
X-Served-By: traefik

Hostname: 725fb61ab07b
IP: 192.168.117.3
Host: whoami.127.0.0.1.nip.io
```

The request returned 200 and carried the `X-Served-By` header, so both the router and the middleware had been created from labels. Figure 5 is the dashboard at that point. It lists the new router with the Docker icon as its provider.

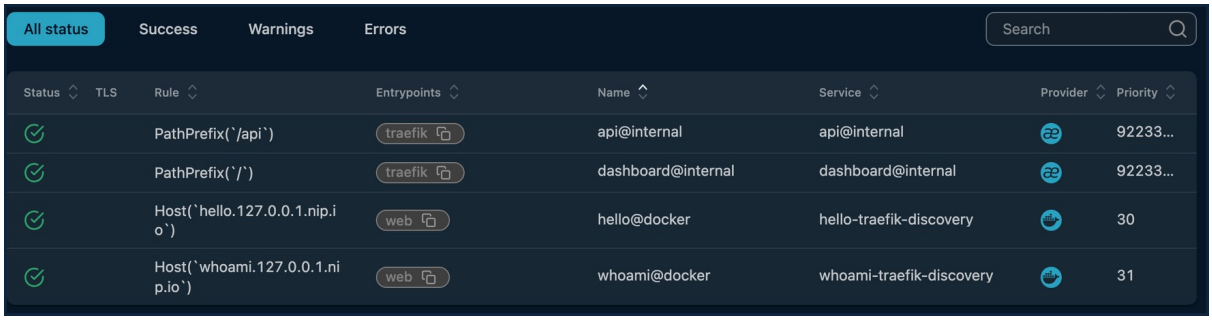



*Figure 5: Dashboard: HTTP routers after two apps were started*

Next I started a second app with its own hostname, and scaled the first one to three replicas. Six requests in a row were answered by all three containers:

```
$ docker compose --profile apps up -d --scale whoami=3 whoami
Hostname: 725fb61ab07b
Hostname: 119b2685f1aa
Hostname: 6fcfeecfb203
Hostname: 119b2685f1aa
Hostname: 6fcfeecfb203
Hostname: 725fb61ab07b
```

Traefik's API reported the three servers behind the `whoami` service, all with status UP, and Figure 6 shows the same in the dashboard.

```
whoami-traefik-discovery@docker  {"http://192.168.117.3:80": "UP",
                                  "http://192.168.117.5:80": "UP",
                                  "http://192.168.117.6:80": "UP"}
```

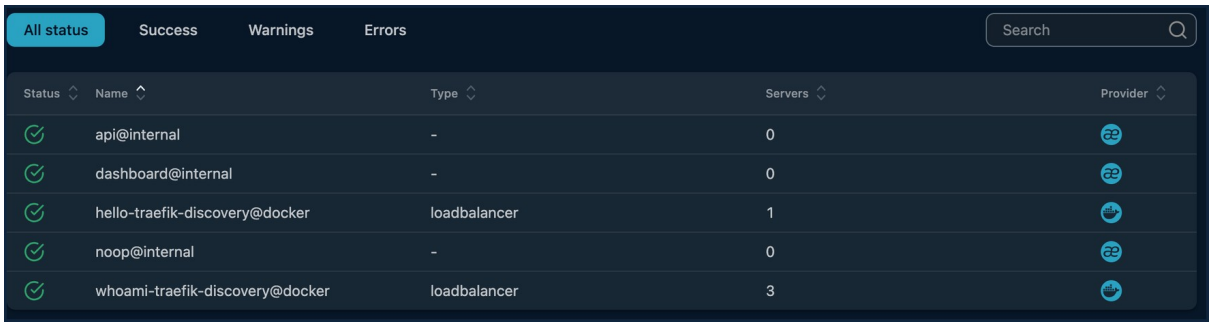



*Figure 6: Dashboard: HTTP services, with three servers behind whoami*

Finally I removed things. Stopping the second app made its hostname return 404 again, and scaling back to one replica left a single server behind `whoami`:

```
$ docker compose --profile apps stop hello
$ curl -s -o /dev/null -w "%{http_code}" -H "Host: hello.127.0.0.1.nip.io" localhost:8000/
404
$ docker compose --profile apps up -d --scale whoami=1 whoami
whoami-traefik-discovery@docker {"http://192.168.117.3:80": "UP"}
```

> Traefik's own container started at 10:24:31 UTC and was never restarted. Every route above was created and removed by starting and stopping containers.

## 6.3 Demonstration 2: automatic HTTPS

Let's Encrypt itself cannot be used from a laptop, because it must reach the server from the internet and the name must be a real public domain. Instead I used Pebble, a small ACME server that Let's Encrypt publishes for testing clients [6]. It follows the same protocol, so Traefik cannot tell the difference. The Compose file starts three containers:

- Pebble, the ACME server.
- Pebble's challenge test server, which answers Pebble's DNS lookups. I told it to point every name at Traefik, which plays the part of the public DNS record that a real domain has.
- Traefik, with a certificate resolver named `pebble` whose `caServer` is the Pebble directory, and the HTTP-01 challenge on an entrypoint that listens on port 5002, the port Pebble connects to.

Traefik starts and registers its ACME provider. No certificate exists yet. Then I started an app whose router asks for the `pebble` resolver. Traefik's log shows the whole exchange:

```
INF Starting provider *acme.Provider
INF Register... providerName=pebble.acme
INF Obtaining bundled SAN certificate. domains=secure.demo.test
INF Validations succeeded; requesting certificates. domains=secure.demo.test
INF Server responded with a certificate. domains=secure.demo.test
```

I did not run any certificate command. To check the result from the client side, I fetched Pebble's root certificate and used it as the only trusted root:

```
$ curl --cacert pebble-root.pem --resolve secure.demo.test:8443:127.0.0.1 \
       https://secure.demo.test:8443/
Hostname: 87391f0879e7
IP: 10.30.50.5

$ openssl s_client -connect 127.0.0.1:8443 -servername secure.demo.test ...
issuer=CN=Pebble Intermediate CA 225f7d
notBefore=Oct 15 10:25:55 2026 GMT
notAfter=Oct 21 10:25:54 2026 GMT
X509v3 Subject Alternative Name: DNS:secure.demo.test
New, TLSv1.3, Cipher is TLS_AES_128_GCM_SHA256
Verify return code: 0 (ok)
```

The same request without the Pebble root fails certificate verification, as it should, because that root is in no trust store. Traefik saved the certificate and account in `acme.json`, which it created with owner-only permissions (`-rw-------`). Figures 7 and 8 show the router and the certificate in the dashboard.

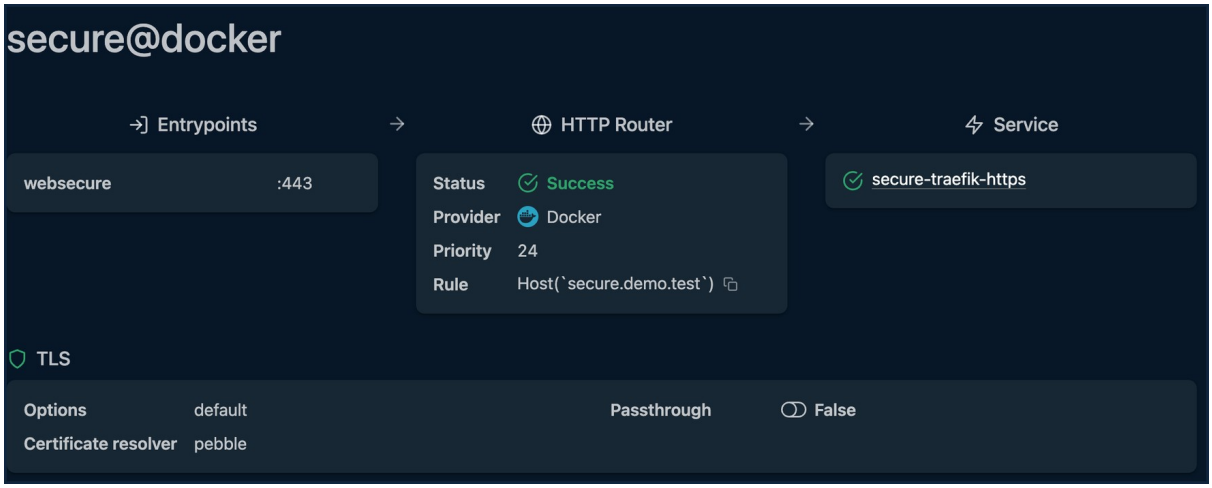



*Figure 7: Dashboard: the secure router with its TLS certificate resolver*

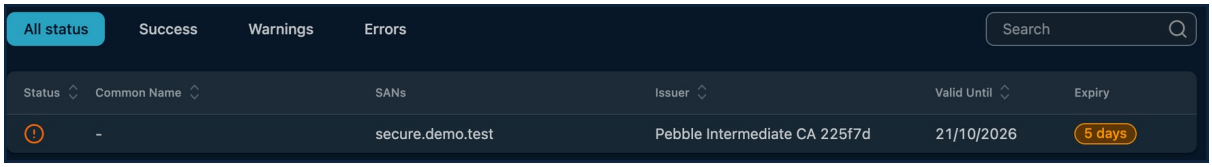



*Figure 8: Dashboard: the certificate Traefik obtained*

The amber badge in Figure 8 says the certificate has five days left. Pebble has two validity profiles, 90 days and 6 days [6], and the certificate in this run had the 6-day one. A certificate that close to expiry is already inside Traefik's 30-day renewal window, so I would expect Traefik to renew it on its next check. I did not wait to see a renewal happen. When I repeated the demonstration a day later, Pebble issued a 90-day certificate in three runs out of three, and I did not find out why the first runs got the short profile.

## 6.4 What the demonstration shows, and what it does not

*Table 5: Scope of the demonstration*

| Shown for real | Not shown |
|---|---|
| Traefik builds, changes and removes routes from container labels while it keeps running | Discovery on Kubernetes, Swarm or other providers |
| Load balancing over replicas, and a middleware applied from labels | Health checks, sticky sessions and other service options |
| A full ACME exchange: account, order, challenge, validation, issue, storage | Let's Encrypt production, public DNS, and a browser-trusted certificate |
| A TLS 1.3 connection that verifies against the issuing root | Certificate renewal, which needs the certificate to reach its renewal window |

## 6.5 Problems I met

### Docker's API version and an older Traefik

Tugboat runs Traefik 3.1. The Docker engine in OrbStack rejects the old API version that Traefik 3.1's Docker client sends, so Tugboat needs an nginx proxy in front of the socket that rewrites the version. Docker 29 raised its minimum API version, and other users met the same error after upgrading [12]. With Traefik 3.7.14 the Docker provider connected directly and no proxy was needed. I did not read the release notes for the exact version that changed this, so I can only say that 3.1 failed and 3.7.14 worked here.

### A start-up warning

Traefik 3.7 logs a warning that `aliasHeadersStrategy` is not configured. Without it, headers such as `X_Auth_User` are forwarded unchanged, and a backend could read them as `X-Auth-User`. The demonstrations leave it at the default. On a real deployment I would set it.

# 7. Traefik inside Tugboat

Tugboat is my DevOps mini project, a self-hosted platform that builds a Git repository into a container and gives it a URL. It uses Traefik in two ways, shown in Figure 9.

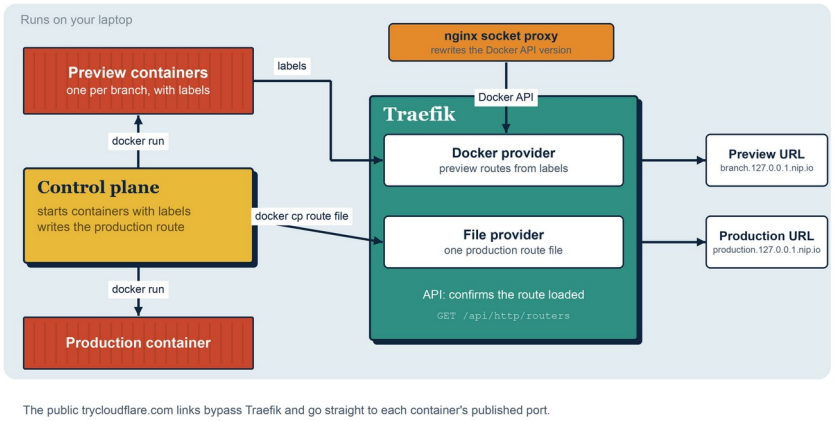



*Figure 9: How Tugboat uses Traefik*

## 7.1 Branch previews through the Docker provider

Tugboat starts each deployment with labels that give it a hostname such as `feature-x.127.0.0.1.nip.io`. Traefik discovers the container and routes to it, with no file written and no restart. An integration test checks that a second labeled container is picked up with no restart, as in the first demonstration.

## 7.2 Production through the file provider

A production swap has to move traffic from the old container to the new one without dropping a request. With labels, the router is removed and added again, and requests can arrive in between. So Tugboat writes one route file that points at the new container, once it has passed a health check. The file is written with `docker cp`, because bind-mount writes are not noticed on macOS. Tugboat then asks Traefik's API whether the route is loaded before it stops the old container. An integration test sends a request every 50 ms while production is promoted twice, and none fail.

## 7.3 What Tugboat does not use Traefik for

Tugboat's local routes are plain HTTP. Its public HTTPS links come from Cloudflare quick tunnels, which connect straight to each container's published port and bypass Traefik. So Tugboat uses Traefik's discovery but not its automatic HTTPS, which would need a real domain.

# 8. Comparison and evaluation

## 8.1 The alternatives

I compared Traefik with four other tools that can sit in front of containers. I ran none of them for this study, so the statements below come from their documentation and announcements.

- nginx is a very widely used web server and proxy that is configured through files.
- Caddy is a web server written in Go whose best-known feature is HTTPS that is on by default.
- HAProxy is a high-performance load balancer configured through a file and a runtime API.
- Envoy is a proxy built to be driven by a control plane, and it is the data plane of several service meshes.

## 8.2 Comparison

*Table 6: Comparison of Traefik with other proxies*

| Tool | Configuration | Container discovery | Automatic HTTPS |
|---|---|---|---|
| **Traefik** | Static settings at start, routes from providers while running | Built in: Docker, Kubernetes, files and others [2] | Built in, with HTTP-01, TLS-ALPN-01 and DNS-01 [3] |
| **nginx** | Configuration files, applied with a reload | None in the core. Community tools generate files from Docker | An official ACME module was announced in 2025 as a preview, HTTP-01 only at launch [7] |
| **Caddy** | A Caddyfile or JSON, with an admin API for changes | A community plugin reads Docker labels [9] | On by default for public names [8] |
| **HAProxy** | Configuration file, plus a runtime API | DNS-based discovery. No Docker label discovery | Experimental ACME support arrived in 3.2 [10] |
| **Envoy** | Dynamic configuration through the xDS APIs from a control plane [11] | Through the control plane, such as a service mesh | Not built in. Certificates are managed by the control plane |

Traefik and Caddy treat certificates as part of the proxy. nginx and HAProxy are adding it. Envoy leaves it to the system around it. For discovery, only Traefik reads the container platform directly in its core.

## 8.3 Evaluation

What worked well in my experiments:

- Discovery behaved as the documentation says. Every change I made to the containers showed up in the routing within the few seconds I waited, and I never restarted Traefik.
- The certificate exchange needed no commands from me. Two flags and one label were enough, and the log showed each step.

- The dashboard made the state visible. A router with a wrong rule or a missing service shows up there with a status, so a label mistake is easy to find.
- The same model of routers, middlewares and services applies to every provider, so what I learned with Docker also applies to files, and I used both.

What I found difficult or risky:

- Labels are long strings with no type checking. A misplaced backtick in a `Host` rule is a silent mistake until the router shows an error.
- Access to the Docker socket is powerful, and the documentation itself recommends a socket proxy [2]. `--api.insecure=true` is a demonstration setting only.
- Traefik and Docker have to agree on an API version. I met this directly: 3.1 stopped working with a newer Docker and 3.7.14 worked. Pinning a Traefik version is ongoing maintenance.
- Automatic HTTPS needs a public name and reachability from the CA. It does not help for a purely internal service unless a private CA or the DNS challenge is used.
- I did not measure throughput or latency, so I make no claim about performance against nginx or HAProxy.

## 8.4 When I would choose which

*Table 7: Choosing a proxy*

| Situation | Choice and reason |
|---|---|
| Containers that start and stop often, on Docker or Kubernetes | Traefik. Discovery and certificates come with the tool |
| A few fixed sites, and a team that knows the tool | nginx. It is mature and every operator knows it |
| A small number of sites where HTTPS must simply work | Caddy. Automatic HTTPS is the default, and a Caddyfile is short |
| Very high traffic, or tuned TCP and HTTP load balancing | HAProxy, which is built for that job |
| A service mesh with a control plane | Envoy, because it is designed to be configured by one |

For my own project the choice was easy. Tugboat's containers come and go on every push, and I wanted routes to follow them without a configuration step. That is the case Traefik is built for.

# 9. Conclusion

Traefik removes two chores from running containers. It builds its routes from the containers that are actually running, so no one edits a proxy file, and it obtains and stores certificates through ACME, so no one requests them by hand. I confirmed both on real software: routes appeared, balanced and disappeared with no restart, and Traefik completed a full ACME exchange and served a certificate that verified against its issuer.

The study has limits. The demonstration used a test CA and not Let's Encrypt, label configuration is easy to get wrong, the Docker socket is a security concern, and Traefik has to be kept compatible with the Docker version it talks to. In Tugboat I use the discovery feature for previews and the file provider for production, and the next step for that project would be to use the HTTPS feature too, with a real domain.

# References

[1] Traefik Labs, "Traefik Proxy documentation," https://doc.traefik.io/traefik/.

[2] Traefik Labs, "Docker provider reference," https://doc.traefik.io/traefik/reference/install-configuration/providers/docker/.

[3] Traefik Labs, "ACME certificate resolvers reference," https://doc.traefik.io/traefik/reference/install-configuration/tls/certificate-resolvers/acme/.

[4] Wikipedia, "Traefik Proxy," https://en.wikipedia.org/wiki/Traefik_Proxy.

[5] R. Barnes, J. Hoffman-Andrews, D. McCarney and J. Kasten, "Automatic Certificate Management Environment (ACME)," RFC 8555, IETF, 2019, https://datatracker.ietf.org/doc/html/rfc8555.

[6] Let's Encrypt, "Pebble," https://github.com/letsencrypt/pebble.

[7] Let's Encrypt, "Native ACME for NGINX," 11 September 2025, https://letsencrypt.org/2025/09/11/native-acme-for-nginx.html.

[8] Caddy, "Automatic HTTPS," https://caddyserver.com/docs/automatic-https.

[9] L. Lorentz, "caddy-docker-proxy," https://github.com/lucaslorentz/caddy-docker-proxy.

[10] HAProxy Technologies, "What is the ACME protocol?" https://cdn.haproxy.com/glossary/what-is-the-acme-protocol.

[11] Envoy Project, "Dynamic configuration (xDS)," https://www.envoyproxy.io/docs/envoy/latest/intro/arch_overview/operations/dynamic_configuration.

[12] Docker Community Forums, "Docker 29 increased minimum API version, breaks Traefik reverse proxy," https://forums.docker.com/t/docker-29-increased-minimum-api-version-breaks-traefik-reverse-proxy/150384.

[13] P. Mistry, "Tugboat," https://github.com/altf4-games/tugboat.